

UNIVERSIDAD AUTÓNOMA DE BAJA CALIFORNIA

FACULTAD DE INGENIERÍA



NOMBRE DEL ALUMNO:

VALLE MARTINEZ KEVIN ALFONSO

Matricula:

1194298

Materia:

Sistemas Operativos

Taller:

Comunicacion de procesos

1. Objetivo

Desarrollar un programa en el lenguaje C que replique el comportamiento de la tubería de comandos `ls -lh | head -1 | wc -c` de Linux, utilizando llamadas al sistema para la creación de procesos y comunicación entre ellos.

2. Desarrollo y Metodología

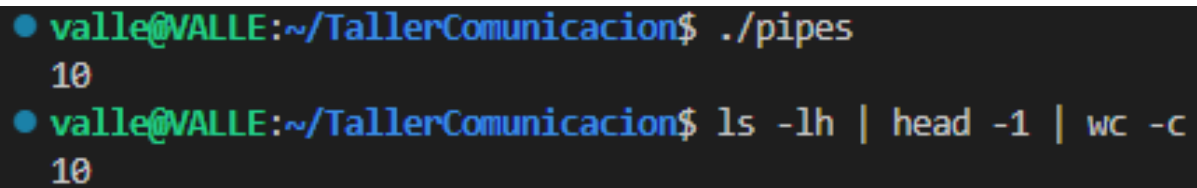
Para lograr la comunicación entre los tres comandos, el programa se estructuró utilizando las siguientes llamadas al sistema (System Calls):

- **pipe()**: Se crearon dos tuberías unidireccionales (pipe1 y pipe2) para conectar las entradas y salidas de los tres procesos.
- **fork()**: Se ejecutó tres veces para crear tres procesos hijos independientes a partir del proceso padre.
- **dup2()**: Se utilizó para redirigir la entrada estándar (stdin) y la salida estándar (stdout) de cada proceso hijo hacia los extremos correspondientes de las tuberías.
- **execvp()**: Permitió que cada proceso hijo reemplazara su imagen de memoria para ejecutar los comandos `ls`, `head` y `wc`.

3. Resultados

Al compilar y ejecutar el programa (llamado `./pipes`), se obtuvo el valor **10**.

Para verificar la exactitud del programa, se ejecutó la misma secuencia de comandos de forma nativa en la terminal (`ls -lh | head -1 | wc -c`). El resultado fue idéntico en ambos casos, comprobando que la comunicación entre procesos se realizó de manera exitosa.



```
● valle@VALLE:~/TallerComunicacion$ ./pipes
10
● valle@VALLE:~/TallerComunicacion$ ls -lh | head -1 | wc -c
10
```

4. Conclusión

Con esta práctica entendí cómo funciona el pipe (`|`) de Linux por dentro. Lo principal que aprendí es que si olvidas cerrar los pipes en el código, el programa se queda trabado para siempre. Ver que me dio el mismo "10" que en la consola me confirma que conecté bien todos los procesos.

5. Anexos

C/C++

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    int pipe1[2]; // Pipe entre ls y head
    int pipe2[2]; // Pipe entre head y wc
    pid_t pid1, pid2, pid3;

    //Crear los pipes
    if (pipe(pipe1) == -1) {
        perror("Error al crear el primer pipe");
        exit(EXIT_FAILURE);
    }
    if (pipe(pipe2) == -1) {
        perror("Error al crear el segundo pipe");
        exit(EXIT_FAILURE);
    }

    //Primer hijo: ls -lh

    pid1 = fork();
    if (pid1 < 0) {
        perror("Error en fork 1");
        exit(EXIT_FAILURE);
    }

    if (pid1 == 0) {

        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        char *args1[] = {"ls", "-lh", NULL};
        execvp(args1[0], args1);

        perror("Error al ejecutar ls");
        exit(EXIT_FAILURE);
    }
```

```

// Segundo hijo: head -1

pid2 = fork();
if (pid2 < 0) {
    perror("Error en fork 2");
    exit(EXIT_FAILURE);
}

if (pid2 == 0) {

    dup2(pipe1[0], STDIN_FILENO);

    dup2(pipe2[1], STDOUT_FILENO);


    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    char *args2[] = {"head", "-1", NULL};
    execvp(args2[0], args2);

    perror("Error al ejecutar head");
    exit(EXIT_FAILURE);
}


// Tercer hijo: wc -c

pid3 = fork();
if (pid3 < 0) {
    perror("Error en fork 3");
    exit(EXIT_FAILURE);
}

if (pid3 == 0) {

    dup2(pipe2[0], STDIN_FILENO);


    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);

```

```
close(pipe2[1]);

char *args3[] = {"wc", "-c", NULL};
execvp(args3[0], args3);

perror("Error al ejecutar wc");
exit(EXIT_FAILURE);
}

// Proceso padre

close(pipe1[0]);
close(pipe1[1]);
close(pipe2[0]);
close(pipe2[1]);

waitpid(pid1, NULL, 0);
waitpid(pid2, NULL, 0);
waitpid(pid3, NULL, 0);

return 0;
}
```